

How Pitch prices a job

The whole credit system, its math, and the reason behind each decision. September 2026.

The short version

- **Users pay for work that actually happened.** Every job is metered in three parts: the AI model's tokens, the machine time (sandbox, renders, recordings), and any paid third-party call such as an AI-generated video clip.
- **The chosen model changes only the token price.** The sandbox and renders cost the same whichever model runs the job.
- **Credits are fine-grained.** One credit = \$0.0025 of billable usage. Usage adds up on the project and is charged in whole credits as it crosses each boundary, so small edits are not rounded up.
- **40 credits or more can start a new job; a job under way runs to zero.** Once started, a job keeps going below 40 until the work is done or the balance reaches zero. A tiny overshoot at the very end is written off, never billed later. At exactly zero, new messages wait for a top-up.
- **A free Discord welcome (750 credits) covers about one finished short video** and can start any film on any model; the job simply stops when the 750 are used.
- **Two jobs can never spend the same credits,** and no balance can go below zero, even when several jobs finish at the same moment. When a job starts, credits are set aside for it (a **hold**, like a card pre-authorization); at the end the user is charged only what the job used and the rest returns.
- **Estimates are guidance, not prices.** The number shown next to each model (Flash about 73 credits a turn, Terra 155, Sol 255, Astra 1,155) comes from the same token prices billing uses. It never blocks a job and is never charged as a flat fee.
- **Users see plain messages.** "You ran out of credits, so this generation stopped. Top up to keep going." Rates, multipliers and internal figures stay in the server logs.

Contents

1. Units: credits and billable usage · 2. What is metered · 3. Model rates · 4. Built-in skill rate · 5. Platform margin · 6. Third-party (provider) costs · 7. Adding up and rounding · 8. The life of a job · 9. Estimates · 10. Plans and what we earn · 11. What users see · 12. What changed in this revision · 13. Open items · 14. Cases tested · Appendix: constants and settings

1. Units: credits and billable usage

There are two units. **Billable usage** is measured in dollars: what the work cost us, adjusted by the rates in sections 3-6. A **credit** is what the user holds and spends. They are linked by one constant:

```
1 credit = $0.0025 of billable usage      (CREDIT_USD)
1 dollar of billable usage = 400 credits
```

A credit is *not* sold at \$0.0025. Plans sell credits for \$0.0128-\$0.025 each (section 10). The gap between what a credit represents and what it sells for is what pays for infrastructure, failed work, payment fees, support and margin.

Why. A small credit keeps prices honest for small work. A quick edit that costs a fraction of a cent adds a fraction to the running total instead of being rounded up to a whole "unit", and plan sizes read as thousands of credits, which leaves room to price cheap and expensive work differently. Keeping the internal rate separate from the retail price means we can change plan prices without re-valuing every balance users already hold.

2. What is metered

Every turn (one request from the user and everything the agent does to answer it) is measured in three parts:

Part	What it is	How it is measured
Model spend	The AI model's input, output and reasoning tokens.	The dollar cost the model provider reports for each message.
Host compute (machine time)	Sandbox tools, browser recording, encoding, rendering, transcription.	Wall-clock seconds inside host actions, at \$0.002 per second, on every model.
Provider spend	Money paid to a third party by a tool, today only AI-generated video clips (Gemini Omni).	Recorded by the tool when the clip comes back: probed length × price per second for that resolution.

They combine into one billable amount for the turn:

```
billable = ( model spend × model rate × skill rate
            + compute seconds × $0.002
            + provider spend × 0.1953 )
            × 1.25 platform margin
credits  = billable ÷ $0.0025
```

Each factor is explained in the next sections. The 0.1953 factor is in section 6.

Only the token part changes with the model. The model rate (section 3) and the built-in skill rate (section 4) multiply model spend and nothing else. Host compute is always seconds × \$0.002 × 1.25, and provider spend is always its own price, whichever model runs the job: 300 seconds of machine time is 300 credits on Flash, on Sol and on Astra.

Why. We charge for what costs us money, and only when it happens. The older scheme priced a project up front by type ("a launch film is 9 credits"), which charged "make me a launch film" and "turn the music down" the same. Metering is fair to users because small requests are cheap, and it protects us because long renders and heavy reasoning cost more. Compute is metered as wall-clock time around each host action because every expensive thing the studio does goes through one, so it is measured in exactly one place.

3. Model rates

A model's price has two parts. Its **token price** is what the provider charges per million tokens; it is already inside the model spend we measure, so a model with dearer tokens costs more automatically. Its **credit rate** (multiplier) is our price on top, applied to token spend only, set per deployment with `STUDIO_MODEL_CREDIT_MULTIPLIERS`.

Model	Token price (\$ per M in / out)	Rate	Typical turn (shown to users)
Gemini 3.8 Flash	0.75 / 3.75	1×	73 credits
Gemini 3.1 Pro	2 / 12	2×	280 credits
Gemma 4 31B / 26B	0 / 0	1× / 0.75×	30 credits
GPT-5.4 mini	0.75 / 4.5	1×	77 credits
GPT-5.4	2.5 / 15	2×	343 credits
GPT-5.5	5 / 30	1.5×	499 credits
Luna / GPT-6 Luna	0.2 / 1.2	0.75×	40 credits
Terra	2 / 12	1×	155 credits
Sol / GPT-6 Sol	4 / 20	1×	255 credits
Astra	10 / 50	2×	1,155 credits

How the typical turn is worked out is in section 9.

Worked example: an Astra turn that spends \$0.20 on tokens and 60 seconds of machine time:

tokens	$\$0.20 \times 2$ (Astra)	= \$0.40
compute	$60 \text{ s} \times \$0.002$	= \$0.12
billable	$(\$0.40 + \$0.12) \times 1.25$	= \$0.65
credits	$\$0.65 \div \0.0025	= 260 credits

The same turn on Flash (1×) is $(\$0.20 + \$0.12) \times 1.25 = \$0.40 = 160$ credits. The 60 seconds of machine time cost 60 credits on both.

Why. Tokens are the only thing that really changes between models: stronger models cost more per token. The sandbox, the browser, the encoder and the renderer are the same machines whichever model is driving them, so charging more for them on a "premium" model would be charging for nothing. The provider's token price passes through as measured, so a price change reaches billing with no code change; the rate is a separate multiplier so our margin per model is one number we control.

4. Built-in skill rate

When a turn loads one of Pitch's built-in production skills (launch film, demo video, slide deck and the others), the **token** part of that turn is billed at 2.5×, on top of the model rate. Compute and provider spend are not multiplied.

```
Flash, $0.10 of tokens, built-in skill loaded:
$0.10 × 1 × 2.5 × 1.25 = $0.3125 → 125 credits
Same turn with no skill:
$0.10 × 1 × 1.25      = $0.125   → 50 credits
```

Why. The skills are the product: the tested pipelines, prompts and tools that turn a request into a finished film or deck. A turn that uses them delivers much more than its raw tokens, so it is priced on the tokens it drives. It applies only to tokens so that a long render inside a skill is not charged 2.5× for machine time. (This rule existed before this revision; it is unchanged, only the user-facing message that mentioned it was removed.)

5. Platform margin

Everything metered is multiplied by **1.25** before it is turned into credits. It is a multiplier, not a fixed fee: a \$0.20 turn becomes \$0.25, not \$0.20 + 25 credits. It is set with `STUDIO_PLATFORM_MARGIN` and cannot go below 1.

Why. A multiplier scales with the work: cheap turns stay cheap and expensive turns carry their share. Keeping it as a separate setting means we can adjust margin without re-valuing credits people already hold and without touching each model's rate.

6. Third-party (provider) costs

Some tools pay an outside company per call. Today that is only **video_generate**, which asks Gemini Omni for an AI-generated clip. It is billed only when a clip is actually produced, from the clip's real length and resolution:

Resolution	Provider price per second (default)
360p	\$0.05
720p	\$0.10
1080p	\$0.15
4K	\$0.40

These per-second prices are placeholders and must be checked against Google's actual Gemini Omni pricing before relying on them. They are set with `OMNI_USD_PER_SECOND`. If the clip length cannot be read, 8 seconds (a typical clip) is billed.

Provider spend is passed through **near cost**, not at the full credit rate. Each dollar is scaled so that it turns into exactly the credits that earn one dollar on our cheapest plan (Max annual, \$0.0128 per credit), then the 1.25 margin applies:

```

provider factor = $0.0025 ÷ $0.0128 = 0.1953
60-second 720p clip:
  provider      60 × $0.10                = $6.00
  billable      $6.00 × 0.1953 × 1.25      = $1.465
  credits       $1.465 ÷ $0.0025           = 585 credits
  worth         585 × $0.0128 (Max annual) = $7.49 (cost $6.00)

```

Why. Model and compute are billed at roughly 5× their cost, which is fine because they are small. Provider costs are large and lumpy: a single minute of generated video costs dollars. At 5× a minute of video would cost users about 3,000 credits, which would make the feature unusable. Passing it through at cost plus the 25% margin, measured against our cheapest credit, guarantees we never lose money on a clip on any plan while keeping the price fair. It is charged per clip that exists, because that is exactly what we are invoiced for.

7. Adding up and rounding

Billable usage is kept in dollars on the project and converted to credits only when charging:

```

project usage += this turn's billable usage
credits earned = floor(project usage ÷ $0.0025)
charge now    = credits earned - credits already charged

```

Example: three edits of \$0.001 billable each are 0.4 credits each. The first two charge nothing; the third crosses 1.2 credits and charges 1. Nothing is lost and nothing is rounded up.

Why. Rounding each turn up would overcharge many small edits; rounding each down would lose money on them. Keeping the running total in dollars and charging only whole credits gives the exact price over the life of a project. The two numbers (usage and credits charged) must always move together; this revision fixed a case where they did not (section 12).

8. The life of a job

What a hold is

A **hold** is credits set aside when a job starts, so nothing else can spend them while it runs. It works like the pre-authorization a hotel or petrol station puts on a card: the money is reserved, not taken, and at the end only what was used is charged.

```
Balance 1,000. Start a 60-second Sol film.  
start  638 set aside      balance 1,000, of which 362 free for anything else  
run    cost measured as the job works  
end    film cost 200      charged 200, the other 438 return → balance 800
```

A hold is **not a price** (the user pays what the job measured, which can be more or less than the hold, or zero), **not a gate** (if it is bigger than the balance it simply holds the whole balance), and **not permanent** (a turn that only answers a question releases it untouched).

Why. Without holds, two jobs started together on 1,000 credits would each believe they had 1,000 to spend, and could spend 1,600 between them. The hold makes the second job see only what the first has not set aside.

Step by step

From the moment someone presses Send to the final charge:

Step	What happens	Why
1. Start check	Starting a new job needs at least 40 credits available. A job already under way is not held to this.	Some cost is only known after it happens (a model call reports its cost when it returns, often 10–50 credits). 40 credits (\$0.10 of usage) is about one step of work. Below that, a job would stop almost at once with nothing to show, and accounts holding 1 credit could get a free first step every time. It applies only to starting, so it never cuts off work that is already running.
2. Hold	A hold is placed: the job's estimate, capped at the balance . Held credits count as spent for every other job.	The hold stops two parallel jobs from each spending the same balance. Capping it at the balance means a user with 231 credits asking for a 60-second Sol film (a 638-credit hold) still starts; the estimate is never a gate.
3. Live check	Every second, the cost so far is compared with hold + balance. If it goes over, the job stops and the user sees one plain message.	This is what actually protects both sides: work continues exactly until the credits run out, instead of being refused up front on a guess.
4. Settle	When the turn ends, the hold is settled for exactly the credits measured , which can be less than the hold or zero. Unused hold returns to the balance.	A hold is a reservation, not a price. The user pays for what ran.
5. Overshoot	If the last step pushed the cost slightly past the balance, the balance goes to 0 and the rest is written off . It is not carried into the next job.	The live check runs between steps, so a small overshoot is unavoidable. Failing the charge used to leave the hold stuck and bill nothing; carrying it forward would surprise the user with a debt. Writing off a few credits is the fair and simple choice, and the 40-credit start minimum keeps the write-off small.
6. Follow-up messages	Later messages on the same project need a balance above 0 (not 40). They are charged directly, down to zero. While a job's hold is still open, messages to it are always accepted.	Below 40 the project keeps going so the work can finish if the credits allow. At exactly 0 a new message is refused: otherwise its first step would run and then be written off, which would make an empty account free to use.
7. No work	A turn that only answers a question or asks the user something releases its hold untouched.	Nothing was produced, so nothing is held.

Hold size

```
hold = max(40, ceil(model estimate × duration ÷ 30 s × video-type factor))  
      capped at the available balance          (duration clamped to 10–300 s)
```

Video type	Factor	Video type	Factor
teaser	0.65	full walkthrough	1.25
feature spotlight	0.9	onboarding tour	1.15
product walkthrough / sales demo	1.0	how-to	1.2
recording edit	0.8	cinematic	1.25
generated video	1.5	motion 3D	1.35

Examples, for a 60-second cinematic film: Flash $\max(40, \text{ceil}(73 \times 2 \times 1.25)) = 183$; Terra 388; Sol 638; Astra 2,888 — or the whole balance if it is smaller.

Why. Longer and more elaborate films take more turns, so the hold grows with duration and type. It only needs to be roughly right, because it is capped at the balance and settled for the actual cost.

Worked example: a Discord welcome account (750 credits)

Asks for a 60 s cinematic film on Sol	hold 638 (fits in 750)
A teaser costs about 630 credits	settles 630 → 120 credits left
A film would cost 1,000 credits	live check stops it at 750 → 0 left
Next message at 0	refused until a top-up

Two jobs at the same time

Every change to an account's credits (placing a hold, settling it, charging a follow-up, or a fixed-price charge such as a browser login session) happens inside one database transaction that first **locks the account**. The balance is read and written under that lock, and credits other jobs are holding count as spent. So when two jobs finish at the same moment, one waits for the other and sees the updated balance.

Why. Without the lock, two turns finishing together could each read "100 left", each charge 80, and leave the account at -60. The real-database tests in section 14 show exactly this failure when the lock is removed.

9. Estimates shown to users

The number shown next to each model prices one **typical turn** exactly the way billing would: the turn's tokens at that model's real token price (the runtime's own price table, the one that turns each message into its cost) and credit rate, plus machine time that is the same on every model, times the margin:

typical turn	50,000 input + 12,500 output tokens, 30 s of machine time
token cost	$(50,000 \times \text{input price} + 12,500 \times \text{output price}) \div 1,000,000$
estimate	$\text{ceil}((\text{token cost} \times \text{rate} + 30 \text{ s} \times \$0.002) \times 1.25 \div \$0.0025)$
Terra	$(\$0.25 \times 1 + \$0.06) \times 1.25 = \$0.3875 \rightarrow 155 \text{ credits}$
Astra	$(\$1.125 \times 2 + \$0.06) \times 1.25 = \$2.8875 \rightarrow 1,155 \text{ credits}$
Gemma	$(\$0 + \$0.06) \times 1.25 = \$0.075 \rightarrow 30 \text{ credits}$

It appears in the model picker and on the pricing page, and it sizes the hold (section 8). It is **never** charged as a minimum and never blocks a job. A model the runtime has no price for is

quoted as a mid-market model (\$2 / \$12), never as free. The built-in skill rate and generated footage are left out because they depend on what a turn does, not on the model.

Known gap: this is one turn, not a whole film. Real finished films take many turns (one recent 60-second launch film used about 870 credits of real work; another project passed 5,500). The typical turn is a setting (STUDIO_TYPICAL_TURN) so it can be calibrated from real projects, per-model medians shown in the admin panel, without a code change.

Why. The estimate used to be 100 credits × the rate, which ignored token prices. That showed Astra as 250, as if it cost only twice Terra, when its tokens are five times dearer; the old "2,500 per 30 seconds" figure was closer, but only because of the fake video price. Pricing the estimate from the same table billing uses means it ranks models correctly by itself, and a price change updates the picker, the pricing page and the holds at once. Machine time is included so no model reads as free: Gemma's tokens cost us nothing, but its sandbox time does.

10. Plans and what we earn

Plan	Price	Credits	Price per credit
Flex (one-time add-on)	\$20	800	\$0.0250
Pro monthly	\$45 / month	2,500 / month	\$0.0180
Max monthly	\$80 / month	5,000 / month	\$0.0160
Pro annual	\$432 / year	30,000 / year	\$0.0144
Max annual	\$768 / year	60,000 / year	\$0.0128

What one credit costs us, against what it earns (on a 1× model):

Plan	Revenue per credit	Margin on model + compute (cost \$0.0020/credit)	Margin on video clips (cost \$0.01024/credit)
Flex	\$0.0250	92%	59%
Pro monthly	\$0.0180	89%	43%
Max monthly	\$0.0160	88%	36%
Pro annual	\$0.0144	86%	29%
Max annual	\$0.0128	84%	20%

Cost per credit is billable ÷ 1.25 for model and compute ($\$0.0025 \div 1.25 = \0.0020), and $\$0.0128 \div 1.25 = \0.01024 for provider spend. Cheaper models (0.75×) lower the model margin slightly; the built-in skill rate and 2× models raise it.

Why. Every row is positive, including the worst case (video clips bought with Max annual credits), so no plan can lose money on any kind of work. The high margin on model and compute covers what metering does not see: idle machines, failed and retried work, storage, payment fees and support.

11. What users see

Situation	Message
Credits run out during a job	"You ran out of credits, so this generation stopped. Top up to keep going."
Too few credits to start (under 40)	"You need more credits to start. Top up to keep going." / on the new-project screen, how many more are needed.
Request refused for credits	"Not enough credits (you have N). Top up to continue..."

Why. Users need to know what happened and what to do, not how the meter works. Earlier messages exposed internal figures ("accrued usage requires 252 credits... includes the 2.5× built-in skill model-usage rate"), which confused people and invited arguments about internals. The exact figures still go to the server logs and the admin panel, where the team can see them.

12. What changed in this revision, and why

Problem 1: charging for video that was never generated

Sol and Astra were listed as "video models" with a fixed provider price of \$7 and \$10 per 30 seconds of the *selected* duration. That price was added as soon as the job made any tool call, whether or not a video was ever generated. Sol and Astra are chat models; generated clips come from Gemini Omni on any model.

60 s on Sol: $\$7 \times (60 \div 30) = \14 charged
(the film was animated in code with library music)

Fix: removed the per-30-second price. Sol and Astra are priced by their token rate like every other model. Real clips are metered per clip (section 6).

Problem 2: the same cost billed twice

The first turn billed that \$14 near cost (about 1,367 credits), but stored the full \$17.50 (with margin) in the project's usage total. The next turn's catch-up (section 7) saw the total and billed it again at the full credit rate: $\$17.50 \div \$0.0025 = 7,000$ credits.

Reported project (60-second launch film on Sol): 7,866 credits charged
of which the \$14 that never happened: $\approx 7,000$ credits
real work: ≈ 870 credits ($\approx \$2.17$ of usage)

Fix: provider spend now enters the usage total at the same near-cost rate it is billed at, so the total and the credits charged always move together. A test replays this case.

Problem 3: a user with 231 credits could not start

The hold for a 60-second Sol film was 2,500 credits (the fake video price), so the job was refused before any work. Even after fixing problem 1, a 60-second cinematic film would ask for 313.

Fix: the hold is capped at the balance. Anyone with 40 or more credits can start; the job runs until the credits run out.

Problem 4: free Discord credits could not start most films

The Discord welcome granted 250 regular credits. With holds sized from the estimate and never capped, a 250-credit account was refused before any work for most 60-second films on a 1× model, nearly anything on a 2× model at 30 seconds or longer, and anything on Sol or Astra (2,500–5,000). And even when a job started, 250 credits was less than half of one finished teaser (about 630).

Fix: holds are capped at the balance (problem 3), and the welcome is raised to **750 credits**, about one finished teaser, so a new user can see a whole video. Plans are unchanged, so the free grant stays well below a paid month (Pro is 2,500).

Problem 5: a message at zero credits ran one step for free

Found while testing this revision. Follow-up messages had no balance check at all, and the new write-off rule forgave the cost of the step they ran. **Fix:** a new message needs a balance above zero; work already running is unaffected.

Problem 6: two charges at once could take a balance below zero

Also found while testing. Follow-up charges and the fixed-price browser session charge read the balance without the account lock. **Fix:** every charge now runs under the lock and counts other jobs' holds as spent (section 8, "Two jobs at the same time").

Problem 7: estimates ignored what tokens cost

The shown estimate was 100 credits × the model's rate, so Astra read as 250 and Sol as 125, the same as Terra, although a typical Astra turn costs about nine times Terra's and Sol about twice.

Fix: the estimate now prices a typical turn from each model's real token price (section 9): Flash 73, Terra 155, Sol 255, Astra 1,155. Holds follow the same numbers. Nothing anyone is charged changed.

Also changed

- The final charge no longer fails when the last step overshoots the balance; it takes the balance to zero and writes off the rest (section 8).
- User-facing messages no longer show internal figures (section 11).
- The pricing page, FAQ and model picker show Sol at about 125 and Astra at about 250 credits, instead of 1,250 and 2,500 per 30 seconds.
- The new-project screen only warns below the 40-credit start minimum, using the same number as the server (a test keeps them equal).
- The Discord welcome is 750 credits, and the app shows the amount the server grants instead of a hard-coded number.

13. Open items

Item	Why it matters
Confirm Gemini Omni prices per second (section 6).	The defaults are placeholders. Too low loses money on clips; too high overcharges.
Refund projects overcharged before this fix (Sol and Astra projects with a selected duration).	Problems 1 and 2 charged thousands of credits for work that did not happen. The reported 60-second film was overcharged by about 7,000 credits.
Fix "About N demo videos" on the pricing page.	It divides plan credits by 120 credits per video; finished videos in testing cost about 630 (teaser) to 2,200 (demo), so the claim is 5-15× too generous.
Announce the Sol/Astra public price change.	The pricing page previously quoted them per 30 seconds.
Confirm Gemma really costs us nothing per token.	The model catalog prices Gemma 4 tokens at \$0, so Gemma turns bill only machine time. Correct if Google does not charge for Gemma; otherwise every Gemma turn is under-billed.
Calibrate the typical turn from real projects (section 9).	Today it is a reasoned guess; real per-model medians would make every estimate match what people spend.
Run the real-database ledger tests in CI (section 14).	They are the only tests that prove the account lock works; they need a disposable Postgres database.

14. Cases tested

Every rule in this document is covered by an automated test. The job-level cases run the real studio worker end to end (the hold, the tool guard, the live check, metering, the real video tool and the real ledger code), with only the database and the AI model replaced by stand-ins. The ledger cases run against a real PostgreSQL database.

Starting a job

Case	Situation	Expected result
A1	1,000 credits, 60 s cinematic Sol film	Hold of 638; balance untouched until the job settles
A2	231 credits, 638-credit hold (the reported case)	Starts; hold shrinks to 231
A3	Exactly 40 credits, Astra	Starts with a 40-credit hold
A4	39 credits	Refused; nothing held or charged
A5	Second job while the first holds all 300 credits	Second job refused
A6	Second message to a job already holding credits	No second hold
A7	Brief unknown up front; first paid tool with 60 credits	Hold placed at the tool, capped at 60
A8	First paid tool with 20 credits	Tool refused before calling the provider; plain message; nothing charged

A Discord welcome account (750 credits)

Case	Situation	Expected result
W1	60 s cinematic film on Sol (hold 638)	Starts with its full 638 hold
W2	60 s film on Astra (hold 2,888)	Starts with a 750 hold
W3	One typical teaser, about 630 credits	Finishes; 120 credits left
W4	Film would cost 1,000	Stopped by the live check; ends at exactly 0; next message refused

During a job

Case	Situation	Expected result
B1	60 s Sol film animated in code, \$0.40 of tokens	200 credits; no \$14 video charge
B2	A real 6 s 720p generated clip on Flash	58 credits (near cost), whichever model asked
B3	The provider refuses the clip	Nothing charged for it
B4	Cost stays within the balance	Job keeps running
B5	Cost passes the balance mid-turn	Job stopped; one plain message with no internal figures
B6	A turn that only answers a question	Hold released; only its tokens (10 credits) charged

Ending a job

Case	Situation	Expected result
C1	Costs 50 against a 638 hold	50 charged; the rest returns
C2	Costs 800 against a 638 hold, balance 1,000	800 charged
C3	Costs 500, only 300 exist	Balance goes to exactly 0; 200 written off, not carried forward
C4	Costs nothing	Hold settled at 0

Follow-up messages

Case	Situation	Expected result
D1	12 credits left, edit costs 50	Runs; charges 12; balance 0
D2	0 credits left, new message	Refused; no step runs
D3	Message while the job's hold is open, 0 available	Accepted
D4	Edit after a generated clip	Only the edit is charged; the clip is never billed twice
D5	Three edits of 0.4 credits each	Charges 0, 0, 1: added up, not rounded up

What the price does not depend on

Case	Situation	Expected result
E1	Same work, 15 s vs 120 s selected	Same charge (100 credits each)
E2	Same token spend on Sol vs Astra	100 vs 200: only the credit rate differs
E4	The same 300 s of machine time on Flash, Sol and Astra	300 credits on each (unit test)
E3	60 s cinematic film on Flash, Sol, Astra	Holds of 183, 638, 2,888, from each model's real token price

Estimates

Case	Situation	Expected result
P1	Pricing page quotes each model	Flash 73, Gemma 30, Terra 155, Sol 255, Astra 1,155
P2	A model priced \$0 per token (Gemma)	Quoted 30 (machine time), never 0
P3	A model with no known price	Quoted as a \$2 / \$12 model, never as free
P4	Rate or typical-turn override in settings	Estimate follows it

The ledger, on a real PostgreSQL database

Case	Situation	Expected result
L1	Two jobs hold the same 300 credits at the same moment	Only one succeeds
L2	231 credits, 313 estimate; and 39 credits	Hold of 231; 39 refused
L3	Settle 500 with 300 available	Exactly 300 charged; balance 0
L4	The same hold settled twice at the same moment	Charged once
L5	Five charges of 40 finish together on 100 credits	Exactly 100 charged; balance 0
L6	Charge 50 while another job holds 70 of 100	Charges 30; the held 70 stays for its job
L7	A hold is released	Every held credit returns
L8	A settlement and a follow-up charge race	Together exactly 100; balance 0
L9	A fixed-price charge races itself on 10 credits	One succeeds; balance 0

How we know the tests can fail

A test that cannot fail proves nothing. Each fix was deliberately undone, one at a time, and the suite was run again:

Fix undone	Cases that caught it
Hold capped at the balance	A2, A3, A5, A7, B5, C3, D1, D2, D3
Overshoot taken to zero	C3
Overshoot written off (not carried forward)	C3, D1
New message needs a balance above zero	D2
Provider cost passed through near cost	B2
Generated clip recorded when made	B2
Account lock on every charge (real database)	L4, L5, L8

Totals for this revision: 30 end-to-end job cases, 9 real-database ledger cases (plus 5 for the Discord welcome grant), and 117 billing, pricing, estimate and metering unit tests, all passing. The full unit suite passes 1,694 of 1,699 tests and the web suite 344 of 344; the 5 failures are slow video-encoding tests that time out under full-suite load and pass on their own, plus one storage test that fails without these changes too.

Appendix: constants and settings

Name	Value	Where
CREDIT_USD	\$0.0025	projects/rates.ts
COMPUTE_USD_PER_SEC	\$0.002	projects/rates.ts
STUDIO_PLATFORM_MARGIN	1.25 (min 1)	env / studio/model-picker.ts
PROVIDED_SKILL_MODEL_MULTIPLIER	2.5 (tokens only)	projects/usage.ts
MIN_REVENUE_USD_PER_CREDIT	\$0.0128 (Max annual)	projects/usage.ts
PROVIDER_USAGE_PER_USD	0.1953 (= 0.0025 ÷ 0.0128)	projects/usage.ts
MIN_BALANCE	40 credits	projects/usage.ts
DEFAULT_TYPICAL_TURN	50k in / 12.5k out tokens, 30 s	studio/model-picker.ts
STUDIO_TYPICAL_TURN	override of the typical turn	env
FALLBACK_TOKEN_PRICE	\$2 / \$12 per M (unknown models)	studio/model-picker.ts
STUDIO_MODEL_CREDIT_MULTIPLIERS	per-model rates (section 3)	env
OMNI_USD_PER_SECOND	360p 0.05 · 720p 0.10 · 1080p 0.15 · 4K 0.40	env / pipelines/video-gen.ts
Live credit check interval	1 second	worker/host.ts
Ledger calls (all under the account lock)	reserveCredits, settleCreditReservation, deductUpTo, deductCredit	packages/db

Paths are under apps/api/src unless noted. Tests: tests/billing.e2e.test.ts (A-E, W), tests/integration/credit-ledger.integration.test.ts (L), tests/usage-channel.test.ts, tests/credits.db.test.ts, tests/video-gen-cost.test.ts.